CPH100

Henry Lee

October 2026

I've included training data for each architecture in section 1: Performance Tables. For the segmentation task, the models would perform too well within a full epoch, and so I opted to include data from the fast exploration mode. All codes can be found on the github repository.

1 Architecture Comparison

Across both classification and segmentation, I saw continual improvements moving from a linear model to an MLP to a CNN, and ultimately a U-Net in the segmentation task. The most striking differences were between linear models and MLPs and between MLPs and CNNs. Each of these steps had large leaps in model performance. Broadly, this indicates that a model with access to local information that can handle nonlinearity is better suited for this task.

In addition, similar model architectures were used for both tasks. My MLP classifier and segmentation MLP both utilize 2 hidden layers of the same sizes, and my CNN models have 3 convolution layers going down, although going back up is very different for the segmentation task.

The ability to handle nonlinear data nearly doubled classifier performance from 34.83% (Table 1) to 60.14% (Table 2) in just 15 epochs. At just 100 steps, the linear and MLP models perform similarly at segmentation. However, in one full epoch, the MLP can shoot ahead of the linear model and reach over 90% validation IoU.

For the classification CNN, I used three convolution layers (C1, C2, C3, in order of depth) with a ReLU after each convolution. The layers had output channels of 64, 128, and 256 respectively. There was only one pool step between C2 and C3. The ReLU of C3 was then flattened passed through a linear layer to be categorized into the 9 classes.

The segmentation CNN was more complicated. It had the same three layer convolutional encoder structure with a ReLU in between each step, but I had two pooling stages, one after C1, and another after C2. These pools were necessary because I used bigger layers (128, 256, and 512) to meet the performance benchmark and the performance gain from less pooling was not worth the increasing in computational complexity. To decode, I used two transposed convolutions

with a 3x3 conv and a ReLU after each. A 1x1 conv is used before passing the tensor into a sigmoid function.

The Tiny U-Net is almost identical in structure to the CNN. The only difference is the addition of skips, where each transposed convolution output is concatenated with the output of the corresponding conv down. This doubled the input channels for my two 3x3 convolution layers that come after the transposed convolution.

2 Performance Analysis

For the sake of brevity, I will focus on the two deliverables: the CNN classifier and the segmentation U-Net, and the tests and choices I made to optimize performance. All of my tests happened during fast exploration mode with 100 steps per epoch, and one epoch with base settings. Modifying hyperparameters had little effect on the performance, and the models took too long to train for hyperparameter tuning to be viable.

Starting with the CNN classifier, after three convolution layers, adding more depth kept performance roughly the same, or lowered by around 5 percentage points. Additionally, I started by tacking on an MLP architecture with hidden layers at the end of three layers in the beginning, but replacing it with one linear layer led to an 11 percentage points improvement. The most variance I saw came from changing the width through adjusting the output channels of the convs, but adjusting width more than a factor of 2 either way led to decreased performance. With kernel sizes, I defaulted to a size of 3x3, but tried a 5x5 C1 to see what would happen. For both CNNs and the U-Net, increasing the kernel of the first layer either had no effect or hindered performance. I also adjusted the number of pooling layers and where I would put them. I found that putting less pools and having them later in the architecture was beneficial for performance, but would increase the parameter space exponentially. Given the images are only 28x28 pixels and the features we care about are relatively small, a 3x3 kernel was big enough to capture most of the patterns necessary to accurately perform the classification task.

What pushed my CNN classifier's validation accuracy was actually data augmentation. My validation loss would bottom out around epoch 3 then rise while training loss kept climbing down, signaling overfitting. By adding data augmentation, I went from overfitting starting from epoch 3, to seeing continual improvements after 15 epochs. If this didn't fix my problem, I would have introduced weight decay but it wasn't necessary.

For the U-Net, adding skip connections did not have a large impact on validation IoU. My biggest improvement in U-Net performance came from changing the decoder structure. I originally had two transpose convolution layers back to back with ReLUs in between. Adding a 3x3 conv after each ReLUs boosted IoU significantly. This improvement worked for the segmentation CNN as well.

3 Performance Tables

Table 1: **Linear classifier, 15 epochs.** 21,177 parameters (flatten, then one fully connected layer, $2352 \to 9$). Best validation accuracy: 37.78% at epoch 10.

Epoch	Train loss	Train acc	Val loss	Val acc
1	1.8904	32.40%	1.8356	36.72%
2	1.8511	33.88%	1.8579	26.76%
3	1.8325	34.14%	1.8485	37.03%
4	1.8101	34.68%	1.7691	35.99%
5	1.7856	34.96%	1.7775	36.02%
6	1.7708	35.04%	1.8009	29.49%
7	1.7632	35.52%	1.8323	30.10%
8	1.7606	35.41%	1.7451	34.29%
9	1.7495	35.80%	1.6905	37.32%
10	1.7439	35.63%	1.7354	**37.78%**
11	1.7363	35.66%	1.7065	37.77%
12	1.7346	35.79%	1.8707	34.30%
13	1.7303	35.99%	1.7197	35.55%
14	1.7299	35.85%	1.7180	33.05%
15	1.7264	35.82%	1.7024	34.83%

Table 2: **MLP classifier, 15 epochs.** 1,338,377 parameters (flatten, then $2352 \to 512 \to 256 \to 9$ with ReLU between layers). Best validation accuracy: 61.07% at epoch 13.

Epoch	Train loss	Train acc	Val loss	Val acc
1	1.3854	47.44%	1.2536	52.42%
2	1.2221	53.76%	1.1975	54.63%
3	1.1676	55.69%	1.1786	55.86%
4	1.1388	56.93%	1.1441	57.25%
5	1.1245	57.36%	1.0991	58.24%
6	1.1093	57.81%	1.1153	57.71%
7	1.0985	58.32%	1.0877	59.26%
8	1.0916	58.60%	1.0967	58.55%
9	1.0796	58.84%	1.0828	58.86%
10	1.0720	59.26%	1.0657	59.60%
11	1.0667	59.80%	1.0621	60.04%
12	1.0574	59.78%	1.0969	58.56%
13	1.0532	60.04%	1.0411	**61.07%**
14	1.0470	60.30%	1.0621	59.81%
15	1.0427	60.37%	**1.0385**	60.14%

Table 3: **CNN classifier without augmentation, 10 epochs.** Same architecture as Table 4 (822,409 parameters); no data augmentation. Validation loss bottoms out at epoch 3 and rises afterward while training loss keeps falling, indicating overfitting. Best validation accuracy: 93.73% at epoch 7.

Epoch	Train loss	Train acc	Val loss	Val acc
1	0.5752	79.09%	0.3448	87.53%
2	0.2714	90.30%	0.2283	91.88%
3	0.1887	93.30%	**0.2184**	92.60%
4	0.1359	95.27%	0.2233	92.99%
5	0.0951	96.65%	0.2572	92.48%
6	0.0718	97.50%	0.2651	93.01%
7	0.0552	98.10%	0.2628	**93.73%**
8	0.0521	98.26%	0.3279	92.57%
9	0.0434	98.57%	0.3085	93.15%
10	0.0456	98.52%	0.3442	92.76%

Table 4: **CNN classifier, 20 epochs.** 822,409 parameters (three 3×3 conv layers with 64/128/256 filters, one max pool after the second, then a single fully connected layer, 50,176 $\rightarrow$ 9); training-set augmentation (flips, 2-pixel shifts, mild color jitter). Best validation accuracy: 96.79% at epoch 16.

Epoch	Train loss	Train acc	Val loss	Val acc
1	0.6820	75.08%	0.3470	88.03%
2	0.3196	88.72%	0.2867	89.55%
3	0.2433	91.41%	0.2128	92.53%
4	0.2000	92.97%	0.2009	92.95%
5	0.1773	93.88%	0.1710	94.03%
6	0.1628	94.40%	0.1805	93.78%
7	0.1498	94.81%	0.1466	95.18%
8	0.1387	95.18%	0.1580	94.83%
9	0.1351	95.31%	0.1241	95.78%
10	0.1262	95.66%	0.1179	96.01%
11	0.1198	95.87%	0.1362	95.24%
12	0.1178	95.92%	0.1290	95.56%
13	0.1112	96.12%	0.1218	95.89%
14	0.1083	96.29%	0.1306	95.54%
15	0.1043	96.38%	0.1242	95.80%
16	0.0988	96.60%	**0.1059**	**96.79%**
17	0.1001	96.51%	0.1153	96.16%
18	0.0969	96.71%	0.1172	96.14%
19	0.0931	96.76%	0.1066	96.56%
20	0.0909	96.86%	0.1176	96.24%

Table 5: **Segmentation architectures, quick discovery.** Each model trained for 100 steps (batch size 32, 3,200 images), combined BCE + Dice loss.

Model	Parameters	Val loss	Val IoU
Linear	1,844,752	0.3829	$47.63\% \pm 30.13\%$
MLP	1,537,552	0.3955	$44.09\% \pm 24.46\%$
Simple CNN (no skips)	2,872,449	0.0429	$93.14\% \pm 16.87\%$
U-Net (skip connections)	3,609,729	0.0461	$\mathbf{93.83}\% \pm \mathbf{18.71}\%$

Table 6: **TinyUNet segmentation, 5 epochs.** 3,609,729 parameters (128/256/512 channels, skip connections at 14×14 and 28×28). Best validation IoU: 99.88% at epoch 4. Final evaluation of the epoch-5 model on all 10,004 validation images: mean IoU $99.83\% \pm 2.95\%$.

Epoch	Train loss	Train IoU	Val loss	Val IoU
1	0.0131	98.19%	0.0057	99.15%
2	0.0021	99.71%	0.0017	99.77%
3	0.0017	99.77%	0.0017	99.82%
4	0.0017	99.80%	**0.0008**	**99.88%**
5	0.0011	99.86%	0.0014	99.82%